

弱标签与引擎评分机制

对话原文美化打印版

用户

#1

```
``powershell
.\.venv\Scripts\python.exe -m engine.build_dataset
````
```

作用：

```
```text
从 engine_detections 表中找两个引擎都有记录的 MD5
合并 360/cm 字段
生成弱标签 weak_label
按 MD5 稳定切分 train/val/test
```
```

输出目录：

```
```text
data\datasets
```
```

文件：

```
```text
train.jsonl
val.jsonl
test.jsonl
```
```

当前数量：

```
```text
train: 87,202
val: 10,981
test: 10,806
```
```

三者用途：

```
```text
train: 分析规律、后续训练候选
val: 调参数
test: 最终验收，不要反复拿来调参数
```
```

注意：这里的标签是弱标签，不是人工金标准。能告诉我怎么合并 360/cm 字段的吗，生成什么样的弱标签，train 训练什么参数，怎么训练，val 怎么调参数

助手

#2

可以。按当前代码实现，逻辑在 build\_dataset.py 和 engine\_store.py。

**\*\*360/cm 字段怎么合并\*\***

`--conflict-only` 会先从 `engine\_detections` 表里找同时存在两个引擎记录的 MD5：

```
```sql
GROUP BY md5
HAVING COUNT(DISTINCT engine) >= 2
```
```

然后对同一个 MD5 的 360/cm 记录做合并：

```
```text
360 -> engine_a
cm  -> engine_b
```
```

合并规则大致是：

```
```text
engine_a_score = 360 的 score
engine_b_score = cm 的 score
```
```

```
engine_a_label / engine_b_label:
score >= 70 -> malicious
score >= 30 -> suspicious
score < 30 -> benign
```
```

文本字段如：

```
```text
app_name
package_name
app_type
platform
control_url
download_url
virus_name
fraud_family
sdk_list
```
```

会按记录顺序取第一个非空值。也就是说如果 360 有值就先用 360，没有再用 cm。

布尔/风险字段是合并增强：

```
```text
fake_app:
任一引擎记录 fake_app 或 impersonation_flag 为真, 就
```

packer:

如果 steady 字段显示不是“未加固/未知/空”，就置 True

signature\_status:

只要有 cert\_md5 / cert\_sha1 / cert\_sha256，就认为 normal

```
```
```

同时会保留原始引擎记录：

```
```json
"engine_records": [
 {
 "engine": "360",
 "score": "0",
 "detect_type": "...",
 "description": "...",
 "find_time": "...",
 },
```

```
{
 "engine": "cm",
 "score": "93.0",
 "detect_type": "...",
 "description": "...",
 "find_time": "..."
}
]
...
```

**\*\*生成什么样的弱标签\*\***

弱标签是三分类：

```
```text
malicious
suspicious
benign
```
```

当前规则是：

- ```
```text
1. fraud_family 非空, 或存在仿冒分类字段 -> malicious

2. 两个引擎最低分都 >= 70 -> malicious
 例：360=90, cm=85

3. 两个引擎最高分 < 30 -> benign
 例：360=0, cm=0

4. 两个引擎分差 >= 35 -> suspicious
 例：360=0, cm=93

5. 最高分 >= 45 -> suspicious
 例：360=20, cm=60

6. 其他 -> benign
```
```

所以这个弱标签不是人工真值，而是“引擎分数 + 业务字段”的启发式标签。它适合做规则调参、样本筛选、后续人工复核，不适合作为最终准确率的绝对标准。

****train 现在训练什么参数****

当前代码里，`train.jsonl` 还没有被真正用于训练模型参数。它目前的定位是：

```
```text
train: 分析规律、后续训练候选
val: 调 WEC/融合参数
test: 最终验收
```
```

现在真正被调的参数在 `evaluate_params.py`：

```
```text
malicious_threshold 恶意阈值
suspicious_threshold 可疑阈值
engine_c_weight Engine C 权重
conflict_boost 双引擎冲突加分
```
```

当前融合公式类似：

```
```text
engine_a = 360_score / 100
engine_b = cm_score / 100
engine_c = 结构化字段估算出来的风险分
```

```
final_score =
(engine_a + engine_b + engine_c *
engine_c_weight)
/
(2 + engine_c_weight)
```
```

然后：

```
```text
final_score >= malicious_threshold -> malicious
final_score >= suspicious_threshold -> suspicious
否则 -> benign
```
```

怎么训练

按当前项目，严格说不是“训练”，而是“调参”。命令是：

```
```powershell
cd C:\Users\啤酒肚\Desktop\工作\test1
.\.venv\Scripts\python.exe -m engine.evaluate_param
```
```

它会自动枚举这些参数组合：

```
```text
malicious_threshold: 0.80, 0.85, 0.90
suspicious_threshold: 0.50, 0.55, 0.60, 0.65
engine_c_weight: 0.6, 0.8, 1.0, 1.2
conflict_boost: 0.0, 0.1, 0.2, 0.3
```
```

输出到：

```
```text
data\eval\best_params.json
data\eval\param_grid_summary.csv
data\eval\review_candidates.csv
```
```

val 怎么调参数

调参目标现在按这个优先级排序：

```
```text
1. risk_recall
2. malicious_recall
3. accuracy
```
```

也就是优先保证“风险样本不要漏掉”，再看恶意召回，最后才看整体准确率。

实际流程建议这样：

```
```powershell
.\.venv\Scripts\python.exe -m engine.evaluate_param
```
```

然后看：

```
```text
data\eval\best_params.json
data\eval\param_grid_summary.csv
```
```

再打开：

```
```text
data\eval\review_candidates.csv
```
```

人工复核前 100 到 300 条，重点看：

```
```text
mismatch: 模型预测和弱标签不一致
engine_conflict: 360/cm 分歧大
high_risk: 高风险样本
boundary: 分数卡在阈值附近
```
```

如果发现 `suspicious` 太多，可以提高：

```
```text
suspicious_threshold
```
```

如果恶意样本漏掉太多，可以降低：

```
```text
malicious_threshold
```
```

如果 360/cm 冲突时希望更依赖 Engine C，就提高：

```
```text
engine_c_weight
conflict_boost
```
```

选好参数后，最后才跑 test:

```
```powershell
.\.venv\Scripts\python.exe -m engine.evaluate_param
```
```

核心点：当前项目还不是“用 train 训练模型，再用 val 调超参”的完整机器学习流程；现在是“用弱标签数据集做规则融合参数搜索”。后续如果要真正训练，可以用 `train.jsonl` 训练一个分类器或 LoRA，但最好先把 `review\_candidates.csv` 里的一批样本人工确认成金标准。

用户

#3

score 是哪里的

助手

#4

`score` 来自导入的 360 / cm Excel 数据，最终写进 SQLite 表：

```
```text
data\mvp.db
表: engine_detections
字段: score
```
```

流程是：

```
```text
360.xlsx / cm.xlsx
↓
engine.import_engine_data
↓
```
```

```

engine_detections.score
↓
engine.build_dataset
↓
engine_a_score / engine_b_score
...

```

在合并时：

```

```python
record_360 = by_engine.get("360")
record_cm = by_engine.get("cm")

```

```

score_360 = score_to_number(record_360["score"]) if
score_cm = score_to_number(record_cm["score"]) if

```

然后：

```

```text
360 的 score -> engine_a_score
cm 的 score  -> engine_b_score

```

所以例如数据里看到：

```

```json
"engine_records": [
 {"engine": "360", "score": "0"},
 {"engine": "cm", "score": "93.0"}
]

```

会变成：

```

```json
"engine_a_score": 0.0,
"engine_b_score": 93.0

```

如果某个引擎没有 score，或者 score 解析失败，
代码默认给：

```

```text

```

	<pre>50.0 ...</pre>  另外, `evaluate_params.py` 里还有一个 `engine_c_score`, 这个不是 Excel 直接来的, 而是脚本根据字段临时算出来的规则分, 比如是否仿冒、是否有 fraud_family、是否有下载链接、360/cm 是否冲突等。
--	---------------------------------------------------------------------------------------------------------------------------------------------------------------

用户   #5	engine_detections.score 的作用
-----------------------------	-----------------------------

助手   #6	`engine_detections.score` 是每个引擎对某个 MD5 的原始风险分数, 是后面所有弱标签和融合判断的基础输入。   它主要有 4 个作用:   1. <b>**生成单引擎标签**</b>   在 engine_store.py 里会把分数转成标签:  <pre>```text score &gt;= 70  -&gt; malicious score &gt;= 30  -&gt; suspicious score &lt; 30   -&gt; benign ```</pre>  所以:  <pre>```text 360 score -&gt; engine_a_label cm score  -&gt; engine_b_label ```</pre>  2. <b>**生成合并样本里的分数字段**</b>   同一个 MD5 的 360/cm 记录合并后:
-----------------------------	------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

```

```text
360 的 score -> input.engine_a_score
cm 的 score  -> input.engine_b_score
```

```

例如：

```

```json
{
  "engine_a_score": 0.0,
  "engine_b_score": 93.0
}
```

```

### 3. \*\*生成弱标签 `weak\_label`\*\*

build\_dataset.py 用两个引擎分数判断弱标签：

```

```text
两个都 >= 70      -> malicious
两个都 < 30       -> benign
分差 >= 35        -> suspicious
最高分 >= 45      -> suspicious
其他              -> benign
```

```

注意：如果有 `fraud\_family` 或仿冒分类，会直接变成 `malicious`，优先级高于 score。

### 4. \*\*参与后续融合评分和调参\*\*

evaluate\_params.py 会把它归一化到 0-1：

```

```text
engine_a = engine_a_score / 100
engine_b = engine_b_score / 100
```

```

再和 `engine\_c\_score` 融合：

```

```text
final_score =
(engine_a + engine_b + engine_c * engine_c_weight)
/
(2 + engine_c_weight)
```

```

|  |                                                                                                                                                                                                                                                                                              |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>最后用阈值判断：</p> <pre>```text final_score &gt;= malicious_threshold -&gt; malicious final_score &gt;= suspicious_threshold -&gt; suspicious 否则                                -&gt; benign ```</pre> <p>一句话：`engine_detections.score` 是 360/cm 给出的风险强度分，既决定单引擎判断，也影响弱标签、冲突识别、最终融合分数和参数调优。</p> |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                     |                       |
|---------------------|-----------------------|
| <p>用户</p> <p>#7</p> | <p>详细解析一下根据什么打分的额</p> |
|---------------------|-----------------------|

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>助手</p> <p>#8</p> | <p>要分清三种“分”：</p> <pre>```text 1. engine_detections.score    360/cm 原始引擎分数，来自 Excel 的 score 列  2. engine_a_score / engine_b_score    360/cm 合并进样本后的分数  3. engine_c_score / final_score    项目自己根据规则和字段算出来的辅助分、融合分 ```</pre> <p><b>**1. engine_detections.score 根据什么打分**</b></p> <p>在当前项目里，`engine_detections.score` 不是代码自己算的，而是导入 360.xlsx / cm.xlsx 时直接读取 Excel 里的 `score` 列。</p> <p>代码映射在 import_engine_data.py：</p> |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
```python
"score": "score"
```
```

也就是说：

```
```text
360.xlsx 里的 score -> engine_detections.score
cm.xlsx 里的 score -> engine_detections.score
```
```

项目只保存和使用这个分数，不知道 360 或 cm 内部具体怎么给出这个分数。内部评分可能来自它们自己的病毒库、规则、行为特征、家族命中、仿冒检测、敏感权限、下载源、签名等，但这些不是当前代码计算的。

**\*\*2. 项目怎么解释这个 score\*\***

当前项目把 `score` 当成 0-100 的风险强度分。

在 engine\_store.py 里规则是：

```
```text
score >= 70 -> malicious    恶意
score >= 30 -> suspicious   可疑
score < 30  -> benign       良性
```
```

例如：

```
```text
360 score = 0      -> benign
cm score = 93      -> malicious
360 score = 50     -> suspicious
360 score = 100    -> malicious
```
```

合并后：

```
```text
360 score -> engine_a_score
cm score  -> engine_b_score
```
```

所以一个样本可能变成：

```
```json
{
  "engine_a_score": 0.0,
  "engine_a_label": "benign",
  "engine_b_score": 93.0,
  "engine_b_label": "malicious"
}
```
```

**\*\*3. weak\_label 根据什么打分\*\***

弱标签不是单看一个 score，而是综合两个引擎分数和业务字段。

规则在 build\_dataset.py：

```
```text
fraud_family 非空
或有仿冒分类字段
-> malicious
```
```

```
两个引擎分数都 >= 70
-> malicious
```

```
两个引擎最高分 < 30
-> benign
```

```
两个引擎分差 >= 35
-> suspicious
```

```
最高分 >= 45
-> suspicious
```

```
其他
...
-> benign
```

举例：

```
```text
360=100, cm=100
-> malicious
```

```
360=0, cm=0
-> benign
```

```
360=0, cm=93
-> suspicious, 因为两个引擎冲突很大
```

```
360=50, cm=60
-> suspicious, 因为最高分 >= 45
```

```
360=10, cm=20
-> benign
```
```

但如果这个样本有：

```
```text
fraud_family = Trojan.Fraud.Sex
```
```

即使分数不高，也会优先标成：

```
```text
malicious
```
```

**\*\*4. engine\_c\_score 根据什么打分\*\***

`engine\_c\_score` 是项目自己根据结构化字段估算出来的，不是 Excel 原始分。

在 evaluate\_params.py 里，初始分是：

```
```text
0.25
```
```

然后根据证据加分：

```
```text
360/cm 分差 >= 35
+ conflict_boost
```

fake_app 为真

+ 0.25

fraud_family / fraud_category_big /

fraud_category_small 非空

+ 0.25

control_url / download_url / control_mailbox /

control_phone 非空

+ 0.12

packer 为真

+ 0.08

描述字段里出现风险关键词

+ 0.18

```
360/cm 任一最高分 >= 85
+ 0.12
```
```

最后限制在：

```
```text
0.0 ~ 1.0
```
```

所以`engine\_c\_score`更像“规则证据分”。它不是训练出来的，而是人为设计的启发式分数。

**\*\*5. final\_score 根据什么打分\*\***

调参时最终分数是三路融合：

```
```text
engine_a = 360_score / 100
engine_b = cm_score / 100
engine_c = 规则证据分
```
```

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>公式：</p> <pre>```text final_score = (engine_a + engine_b + engine_c * engine_c_weight) / (2 + engine_c_weight) ```</pre> <p>然后用阈值判断：</p> <pre>```text final_score &gt;= malicious_threshold -&gt; malicious final_score &gt;= suspicious_threshold -&gt; suspicious 否则                                -&gt; benign ```</pre> <p>一句话总结：</p> <pre>```text engine_detections.score 是 360/cm Excel 里已有的原始数据 项目不负责计算它，只负责解释、合并和使用它。 weak_label 用 360/cm 分数 + fraud_family/仿冒字段生成 engine_c_score 是项目根据 fake_app、fraud_family、User final_score 是 360 分、cm 分、Engine C 分的加权融合结果 ```</pre> |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                     |                          |
|---------------------|--------------------------|
| <p>用户</p> <p>#9</p> | <p>弱标签是哪个代码定义的，怎么定义的</p> |
|---------------------|--------------------------|

|                      |                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <p>助手</p> <p>#10</p> | <p>弱标签在这个文件里定义：</p> <p>build_dataset.py</p> <p>核心函数是：</p> <pre>```python def weak_label(sample: dict[str, Any]) -&gt; str: ```</pre> |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------|

定义逻辑如下：

```
```python
def weak_label(sample: dict[str, Any]) -> str:
    score_a = float(sample.get("engine_a_score", 5))
    score_b = float(sample.get("engine_b_score", 5))
    scores = [score_a, score_b]
```

```
has_fraud_family = bool(str(sample.get("fraud_
```

```
has_impersonation_category = any(
    str(
        record.get("impersonation_l1", "")
        or record.get("impersonation_l2", "")
        or record.get("impersonation_l3", "")
    ).strip()
    for record in sample.get("engine_records",
)
```

```
if has_fraud_family or has_impersonation_cate
    return "malicious"
```

```
if min(scores) >= 70:
    return "malicious"
```

```
if max(scores) < 30:
    return "benign"
```

```
if abs(score_a - score_b) >= 35:
    return "suspicious"
```

```
if max(scores) >= 45:
    return "suspicious"
```

```
return "benign"
```

```
```
```

翻译成规则就是：

```
```text
```

1. 如果 fraud_family 非空
-> malicious

2. 如果 engine_records 里存在 impersonation_l1 / 12
-> malicious

3. 如果 360 和 cm 两个分数都 >= 70
-> malicious

4. 如果 360 和 cm 两个分数都 < 30
-> benign

5. 如果 360 和 cm 分差 >= 35
-> suspicious

6. 如果两个分数里最高分 >= 45
-> suspicious

7. 其他情况
-> benign
```

其中：

```
```text
```

```
engine_a_score = 360 的 score  
engine_b_score = cm 的 score  
```
```

弱标签生成后写进 JSONL：

```
```json
```

```
{  
  "id": "MD5",  
  "split": "train",  
  "weak_label": "suspicious",  
}
```

"input": {...},
"target": {
 "verdict": "suspicious",
 "note": "weak label generated from engine scores and business fields
label"
}
}
},
...

所以弱标签本质是“规则标签”，不是人工金标准。它用来做调参、初步训练候选和人工复核筛选。

用户

#11

这是什么 fraud_family, 这个呢, final_score >= malicious
final_score >= suspicious_threshold -> suspicious
否则 -> benign, 这个是
2. malicious_recall
3. accuracy, 是这样吗, 我先使用训练数据训练好各个参数, 标签, 只对比结果, 看准确率, 如果准确率不够再调参吗

助手

#12

`fraud\_family` 是样本的“涉诈/恶意家族”字段, 来自导入的 360/cm 数据记录, 不是模型训练出来的。

例如数据里可能有:

```json  
"virus\_name": "Trojan.Fraud.Sex",  
"fraud\_family": "J-色情视频-46b48d783"  
```

含义大概是: 这个 APP 被某个引擎或业务规则归到了某个恶意/涉诈家族里。当前代码认为: 只要
`fraud\_family` 非空, 就是强风险信号, 所以弱标签直接给:

```text

原文打印版 · 未整理改写

```
malicious
```
```

也就是：

```
```python
if has_fraud_family:
 return "malicious"
```
```

你问的这个：

```
```text
final_score >= malicious_threshold -> malicious
final_score >= suspicious_threshold -> suspicious
否则 -> benign
```
```

意思是：系统最终会算出一个 `final\_score`，范围大概是 `0 ~ 1`。然后用两个阈值切三类。

比如当前较好的参数是：

```
```text
malicious_threshold = 0.80
suspicious_threshold = 0.50
```
```

那么：

```
```text
final_score >= 0.80
-> malicious 恶意
```
```

```
0.50 <= final_score < 0.80
-> suspicious 可疑
```

```
final_score < 0.50
-> benign 良性
```
```

举例：

```
```text
final_score = 0.91 -> malicious
final_score = 0.63 -> suspicious
final_score = 0.22 -> benign
```
```

---

这个排序：

```
```text
1. risk_recall
2. malicious_recall
3. accuracy
```
```

是对的。当前 `evaluate\_params.py` 里就是优先按这个顺序选参数：

```
```python
results.sort(
    key=lambda item: (
        item["risk_recall"],
        item["malicious_recall"],
        item["accuracy"]
    ),
    reverse=True
)
```

含义是：

```
```text
risk_recall:
真实为 suspicious 或 malicious 的样本，有多少被系统识别
重点是不要漏掉风险样本。
```

malicious\_recall：

真实为 malicious 的样本，有多少被系统识别成 malicious。

重点是不要漏掉明确恶意样本。

accuracy：

三分类整体准确率。

...

为什么先看 `risk\_recall`? 因为恶意 APP 研判里, “漏报风险” 通常比 “多报一些可疑” 更严重。

---

你最后那段理解需要修正一下。

不是:

```
```text
先用 train 训练好参数
然后不用 val 标签, 只看结果
准确率不够再调参
```
```

更准确的流程是:

```
```text
1. train: 用来训练模型, 或者分析规则
2. val: 用来调参数, 必须使用 val 的标签来计算准确率/否
3. test: 最终验收, 只跑一次或少量几次, 不反复调
```
```

如果 “不使用 val 数据集的标签”, 你就没法计算:

```
```text
accuracy
risk_recall
malicious_recall
precision
confusion matrix
```
```

因为这些指标都必须知道 “预测结果” 和 “标签” 是否一致。

正确理解是:

```
```text
训练阶段:
可以使用 train 的标签训练模型。
```

调参阶段:

使用 val 的输入做预测；
 使用 val 的标签计算指标；
 根据指标和人工复核结果调参数。

测试阶段：

参数固定后，使用 test 的输入做预测；
 使用 test 的标签计算最终指标；
 不要再根据 test 结果反复改参数。

...

但在你当前项目里，还有一个关键点：

****现在还没有真正用 `train.jsonl` 训练模型。****

目前 `evaluate\_params.py` 做的是“参数搜索”，
 不是机器学习训练。它直接在 `val.jsonl` 上枚举这
 些参数：

```
```text
malicious_threshold
suspicious_threshold
engine_c_weight
conflict_boost
```
```

然后用 `val.jsonl` 里的 `weak\_label` 评估哪组参数
 最好。

也就是说，当前流程是：

```
```text
build_dataset 生成 weak_label
evaluate_params 用 val 的 weak_label 选融合参数
```
```

不是：

```
```text
train 训练模型参数
val 验证模型参数
```
```

如果后续你真的要“训练模型”，可以用的标签是：

```
```text
train.jsonl 里的 weak_label
```
```

或者等价地：

```
```text
target.verdict
```
```

例如：

```
```json
{
 "weak_label": "suspicious",
 "target": {
 "verdict": "suspicious"
 }
}
```
```

但要注意：这是弱标签，不是人工金标准。最好做法是：

```
```text
1. 先用 weak_label 做初始训练
2. 从 review_candidates.csv 里挑样本人工复核
3. 形成一批人工 gold label
4. 再用人工标签校准模型和阈值
```
```

一句话总结：

```
```text
fraud_family 是涉诈/恶意家族字段，非空时当前规则直接认定
final_score 是最终融合风险分，用阈值切成 malicious /
val 调参时必须用 val 的标签算指标，但不能拿 val 去训练
当前项目还没有真正训练 train，只是在 val 上调融合规则
```
```

用户

#13

这个不对吧，4. engine_c_score 根据什么打分
engine_c_score 是项目自己根据结构化字段估算出来的，不是 Excel 原始分。

在 evaluate_params.py 里，初始分是：

text

0.25

然后根据证据加分：

text

```
360/cm 分差 >= 35  
+ conflict_boost
```

fake_app 为真

+ 0.25

fraud_family / fraud_category_big /

fraud_category_small 非空

+ 0.25

control_url / download_url / control_mailbox /

control_phone 非空

+ 0.12

packer 为真

+ 0.08

描述字段里出现风险关键词

+ 0.18

```
360/cm 任一最高分 >= 85  
+ 0.12
```

最后限制在：

text

0.0 ~ 1.0

所以 engine_c_score 更像“规则证据分”。它不是训练出来的，而是人为设计的启发式分数。引擎 c

的分数不是 Engine C：专门针对冲突样本，调用双模型辩论与多智能体协同能力，的出来的吗

助手

#14

你说得对。这里需要区分两个地方的 `Engine C`：

```
```text
真实研判流程里的 Engine C
= 多智能体证据块 + 双模型辩论 + 仲裁器得出的分数
```
```

evaluate_params.py 里的 engine_c_score
= 为了批量调参，临时用结构化规则近似出来的
Engine C 分数
...

也就是说，我前面那段说的是
**`evaluate\_params.py` 离线调参脚本里的近似
engine_c_score`**，不是页面实际研判时完整
Engine C 的真实生成流程。

真实 Engine C 在 pipeline.py 里，流程是：

```
```text
sample
↓
extract_iocs()
↓
run_agents()
↓
四个专家智能体打证据分
↓
debate()
↓
模型甲 / 模型乙辩论
↓
arbiter 生成 Engine C score
↓
wec_decision() 和 360/cm 融合
```
```

四个智能体是：

```
```python
```

```
run_agents(sample, iocs):
 return [
 static_analysis_agent(sample),
 threat_intel_agent(sample),
 impersonation_agent(sample),
 business_label_agent(sample),
 ...]
```

它们分别看：

```
```text  
static_analysis_agent:  
签名状态、权限、加固/壳等静态特征
```

threat_intel_agent:
control_url、download_url、邮箱、电话、IOC、
fraud_family

impersonation_agent:
fake_app、app_name、package_name、
brand_similarity

business_label_agent:
virus_name、fraud_family 里的业务/家族标签
...

然后 `debate()` 会产生：

```
```text  
model_a_score: 保守复核模型分
model_b_score: 风险优先模型分
arbiter.score: 仲裁后的 Engine C 分
```
```

如果开启本地 Qwen：

```
```text  
MALAPP_USE_LOCAL_QWEN=1
```
```

就会调用本地模型做甲/乙辩论。

如果没开启，代码会用规则兜底：

```
```text
rule_fallback
```
```

最终 Engine C 的仲裁分是：

```
```python
c_score = clamp((model_a_score + model_b_score + m
```
```

这里的 `max\_block.score` 是四个智能体里最高的证据块分数。

所以真实 Engine C 不是简单的：

```
```text
fake_app +0.25
fraud_family +0.25
```
```

真实流程是：

```
```text
多个专家智能体先各自打证据分
模型甲/模型乙基于证据块辩论
仲裁器综合模型甲、模型乙、最强证据块
得到 Engine C 分数
```
```

而 `evaluate\_params.py` 为什么不用真实 Engine C？

因为 `val.jsonl` 有一万多条，如果每条都跑本地 Qwen 双模型辩论，会非常慢。代码注释里也写了，它是：

```
```text
Fast Engine-C-like score for parameter search.
This avoids running local Qwen across 10k+ validat
```
```

所以调参脚本里的 `engine\_c\_score` 是 “Engine

| | |
|--|--|
| | <p>C-like score” ，也就是近似版、加速版。</p> <p>正确表述应该是：</p> <div><pre>```text 页面/单样本研判： Engine C = 多智能体 + 双模型辩论 + 仲裁 ```</pre></div> <div><pre>```text 批量 val 调参： engine_c_score = 为了快速评估参数，用规则近似 Engine ```</pre></div> <p>你指出的是对的。前面我把“调参脚本里的近似 Engine C 分数”说得像完整 Engine C，这是不严谨的。</p> |
|--|--|

| | |
|----------------------|-----------------------------------|
| <p>用户</p> <p>#15</p> | <p>那写的近似版能反映真实情况吗，准确率不会具有代表性吧</p> |
|----------------------|-----------------------------------|

| | |
|----------------------|---|
| <p>助手</p> <p>#16</p> | <p>你这个质疑是对的：**近似版`engine_c_score`不能严格代表真实 Engine C，所以它跑出来的准确率也不能直接代表真实系统准确率。**</p> <p>它只能代表：</p> <div><pre>```text 在“规则近似 Engine C”的条件下，这组融合参数和 weak_la ```</pre></div> <p>不能代表：</p> <div><pre>```text 真实多智能体 + 双模型辩论 Engine C 的最终准确率 ```</pre></div> |
|----------------------|---|

原因有三个：

1. ****近似版没有跑模型甲/模型乙****

真实 Engine C 会走：

```
```text
专家智能体证据块
-> 模型甲保守复核
-> 模型乙风险优先
-> 辩论/反驳
-> 仲裁
```
```

而 `evaluate\_params.py` 只是根据字段加分：

```
```text
fake_app
fraud_family
download_url
packer
engine conflict
```
```

它没有理解描述文本，也没有真正做辩论。

2. ****近似版和真实 Engine C 的分布可能不同****

真实 Engine C 输出可能偏保守，也可能偏激进。比如同样一个样本：

```
```text
近似版 engine_c_score = 0.85
真实 Engine C score = 0.62
```
```

或者反过来：

```
```text
近似版 engine_c_score = 0.45
真实 Engine C score = 0.80
```
```

如果分数分布不同，那么用近似版调出来的：

```
```text
engine_c_weight
malicious_threshold
suspicious_threshold
```
```

放到真实 Engine C 上未必最优。

3. ****当前准确率对的是 weak_label，不是人工金标准****

即使近似 Engine C 完全一致，它对比的也是：

```
```text
weak_label
```
```

而不是人工标注。弱标签本身来自 360/cm 分数和业务字段，有偏差。

所以当前 `evaluate\_params.py` 的结果只能叫：

```
```text
离线弱标签一致性评估
```
```

不能叫：

```
```text
真实模型准确率
```
```

比较严谨的做法应该是两阶段。

****第一阶段：快速粗调****

用现在的近似版跑全量 `val.jsonl`，快速筛选一个大致参数范围：

```
```text
malicious_threshold
suspicious_threshold
engine_c_weight
```
```

```
conflict_boost
```
```

这个阶段只解决：

```
```text
哪些参数明显不合理？
风险召回大概能不能兜住？
哪些样本值得人工复核？
```
```

它适合快，但不适合当最终结论。

**\*\*第二阶段：真实 Engine C 小样本验证\*\***

从 `val.jsonl` 抽样，比如：

```
```text
500 条
1000 条
```
```

尤其要覆盖：

```
```text
360/cm 冲突样本
边界分数样本
高风险样本
弱标签和预测不一致样本
良性样本
```
```

然后真的跑：

```
```text
pipeline.judge()
```
```

也就是完整 Engine C：

```
```text
多智能体 + 双模型辩论 + 仲裁
```
```

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>再用这些结果重新评估参数。</p> <p>这样得到的指标才更接近真实系统表现。</p> <p>---</p> <p>现在代码里缺的就是一个类似：</p> <pre>```powershell python -m engine.evaluate_real_engine_c --dataset ```</pre> <p>这个脚本应该做：</p> <pre>```text 1. 读取 val.jsonl 2. 对每条 input 调 pipeline.judge() 3. 拿 judge() 里的真实 engine_c score / final_score 4. 和 weak_label 或人工 label 对比 5. 输出 accuracy / recall / confusion matrix ```</pre> <p>如果后续有人工标签，最好对比人工标签，不要只对比 weak_label。</p> <p>---</p> <p>结论：</p> <pre>```text 近似版能反映一部分结构化风险规律，但不能完整反映真实 Engine C 它适合快速粗调，不适合作为最终准确率。 当前 evaluate_params.py 的准确率不具有最终代表性，只能作为参考 最终应该用真实 Engine C + 人工复核标签做验证。 ```</pre> |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                      |                                              |
|----------------------|----------------------------------------------|
| <p>用户</p> <p>#17</p> | <p>什么情况下 review_candidates.csv：建议人工复核的样本</p> |
|----------------------|----------------------------------------------|

## 助手

#18

`review\_candidates.csv` 是 evaluate\_params.py 自动挑出来的“值得人工看一眼”的样本。

触发条件在函数：

```
```python
write_review_candidates()
```
```

只要满足下面任意一种情况，就会进入 `review\_candidates.csv`。

**\*\*1. mismatch：预测结果和弱标签不一致\*\***

```
```text
pred != weak_label
```
```

例如：

```
```text
weak_label = suspicious
pred = benign
```
```

这种最值得看，因为可能是：

```
```text
弱标签错了
系统规则错了
阈值不合适
字段缺失或异常
```
```

**\*\*2. engine\_conflict：360/cm 分歧很大\*\***

```
```text
abs(engine_a_score - engine_b_score) >= 0.35
```
```

注意这里的分数已经除以 100，所以等价于：

```
```text
360/cm 原始分差 >= 35
```
```

例如：

```
```text
360 = 0
cm = 93
```
```

这种就是典型冲突样本，正是 Engine C 要重点仲裁的对象。

**\*\*3. high\_risk：高风险样本\*\***

满足任意一个：

```
```text
final_score >= malicious_threshold
或 fake_app 为真
或 fraud_family 非空
```
```

也就是：

```
```text
系统已经判高危
或存在仿冒标记
或存在涉诈/恶意家族
```
```

这种需要人工确认，防止高风险误报或漏掉关键家族。

**\*\*4. boundary：边界样本\*\***

```
```text
suspicious_threshold - 0.05 <= final_score <= mali
```
```

比如参数是：

```
```text
```

```
suspicious_threshold = 0.50
malicious_threshold = 0.80
```
```

那么边界范围大概是：

```
```text
0.45 ~ 0.85
```
```

这些样本靠近阈值，稍微调一下参数就可能从：

```
```text
benign -> suspicious
suspicious -> malicious
```
```

所以适合人工看，用来判断阈值是不是合理。

---

进入 CSV 后，还会按优先级排序：

```
```text
mismatch          权重 4
engine_conflict    权重 3
high_risk          权重 2
boundary          权重 1
```
```

优先级越高越靠前。

所以最靠前的一般是：

```
```text
预测和弱标签不一致
并且 360/cm 冲突
并且高风险
并且靠近阈值
```
```

这类样本最适合人工复核。

一句话总结：

```
```text
review_candidates.csv 不是随机样本，而是系统自动挑出的“最可能暴露规则问题、弱
题、引擎冲突问题”的样本。
```
```

用户

#19

是使用 val 的真实标签和训练出来的标签不一致吗

助手

#20

不是“真实标签和训练出来的标签不一致”。

当前项目里更准确是：

```
```text
review_candidates.csv 里的 mismatch
= val.jsonl 里的 weak_label
和 evaluate_params.py 当前参数预测出来的 pred
不一致
```
```

也就是：

```
```text
weak_label != pred
```
```

这里没有人工真实标签参与。

例如 `val.jsonl` 某条是：

```
```json
{
  "weak_label": "suspicious",
  "input": {
    "engine_a_score": 0.0,
    "engine_b_score": 93.0
  }
}
```
```

`evaluate\_params.py` 用当前参数算出：

```
```text
pred = malicious
```
```

那么就会记为：

```
```text
mismatch
```
```

含义是：

```
```text
弱标签说  suspicious
当前融合规则预测  malicious
两者不一致，建议人工复核
```
```

但不能直接说谁错，因为：

```
```text
weak_label 不是人工金标准
pred 也不是训练模型真值
```
```

它只是提示：这条样本值得人工看。

---

你说的“val 的真实标签”在严格机器学习里应该是人工标注的 gold label。但你当前 `val.jsonl` 里没有真正人工金标准，只有：

```
```text
weak_label
```
```

所以当前流程是：

```
```text
val 输入样本
↓
当前参数规则预测 pred
```
```

|  |                                                                                                                                                                                                                                                                                                                                                                                                      |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div><div>↓<br/>和 <code>val.weak_label</code> 对比<br/>↓<br/>不一致 -&gt; <code>review_candidates.csv</code><br/>...</div><div>不是:</div><div><div>```text<br/>val 真实人工标签<br/>↓<br/>和训练模型标签对比<br/>```</div><div>更准确地说:</div><div><div>```text<br/>review_candidates.csv 是“弱标签和当前规则预测不一致”<br/>```</div><div>人工复核后，你才可以给它们补上真正的:</div><div><div>```text<br/>gold_label<br/>```</div></div></div></div></div> |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                        |                               |
|----------------------------------------|-------------------------------|
| <div><div>用户</div><div>#21</div></div> | <div>将上面的对话打印成文档，PDF 格式</div> |
|----------------------------------------|-------------------------------|

|                                        |                                                                                                                                                                                                          |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>助手</div><div>#22</div></div> | <div><div>已生成 PDF:</div><div>弱标签与引擎评分机制_对话整理.pdf</div><div>同时保留了 DOCX 原稿:</div><div>弱标签与引擎评分机制_对话整理.docx</div><div>校验结果: PDF 已成功导出，文件大小约 346 KB;</div><div>Word 分页统计为 5 页。LibreOffice 渲染器本机未</div></div> |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                       |
|--|---------------------------------------|
|  | 找到，所以改用 Microsoft Word COM 完成 PDF 导出。 |
|--|---------------------------------------|

|                              |                              |
|------------------------------|------------------------------|
| <div>用户</div> <div>#23</div> | 不对，不要整理，直接将上面所有的内容打印成 PDF 即可 |
|------------------------------|------------------------------|